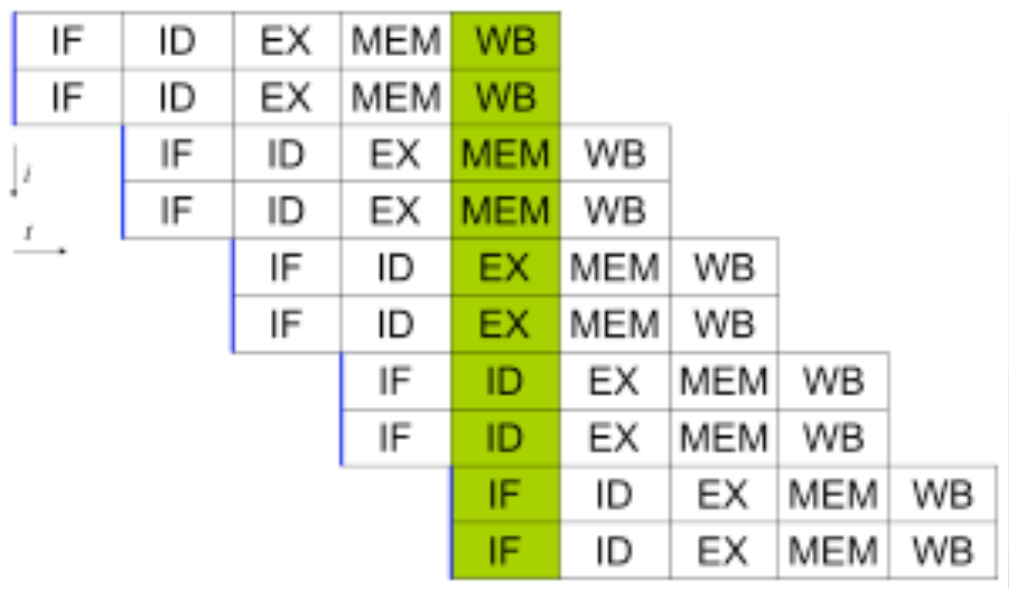
Electronics and
Computer Science

UNIVERSITY OF
Southampton

21. Pipelining (6)

Superscalar

- Even with an optimally used pipeline, Instructions Per Cycle (IPC) = 1.
- Superscalar means IPC > 1… How?
- In general, more than one pipeline, or part of a pipeline.

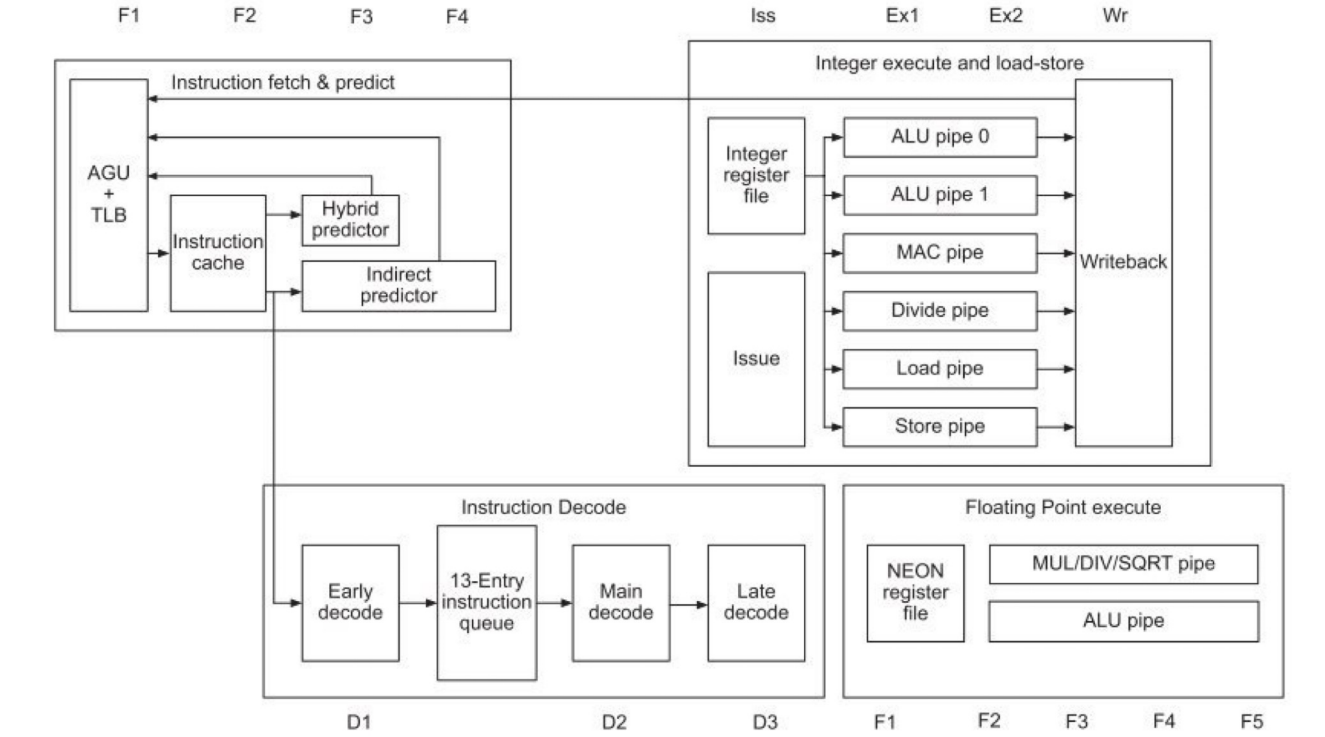


http://en.wikipedia.org/wiki/Superscalar

Superscalar

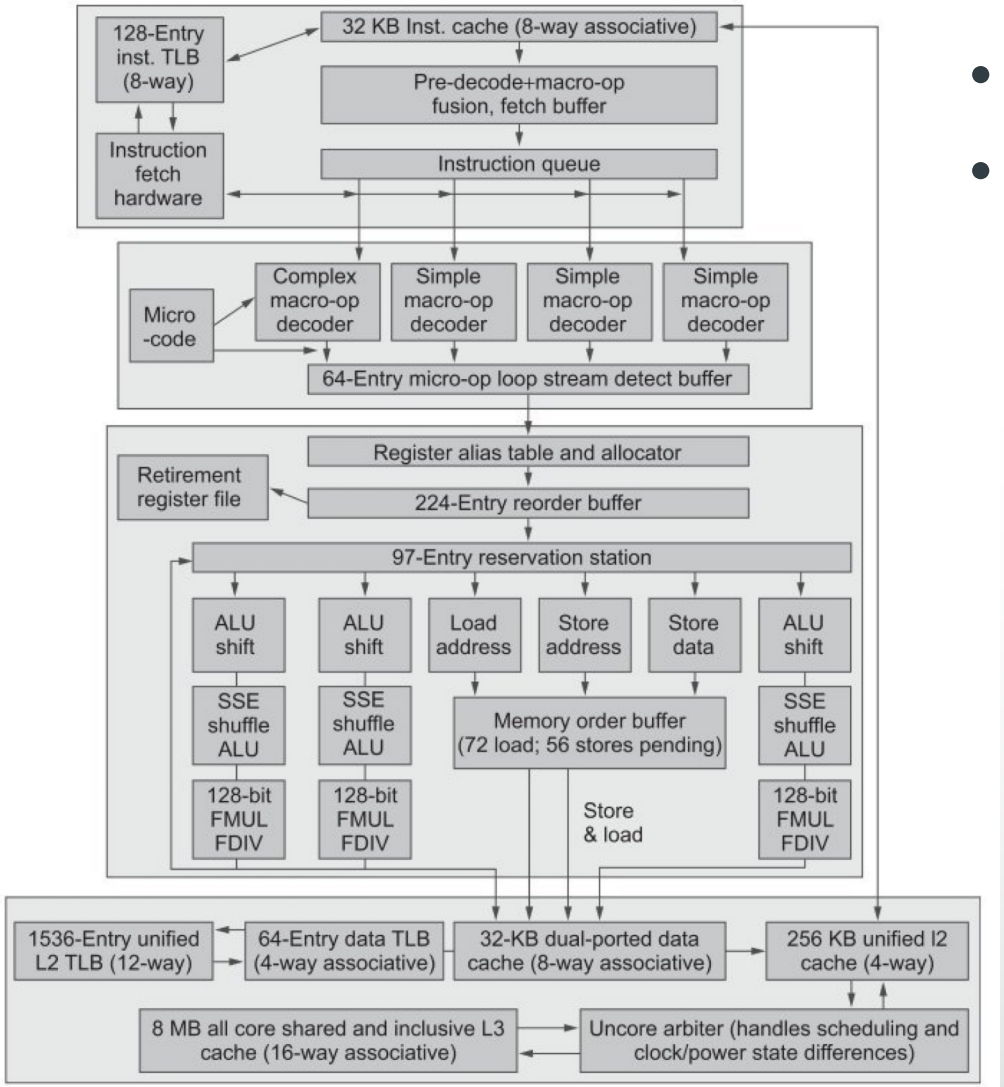
- ARM A-8 has two instruction issue; two ALU pipelines (symmetric)
- Could have asymmetric pipelines (integer and floating point)
- BUT can only execute instructions in parallel, if there is no dependency between them
- So far, we have assumed in-order execution…

Arm Cortex-A53



- Integer pipeline has 8 stages, multiple EX units
- Floating point has 5 stages + 5 for fetch and decode = 10

Intel i7

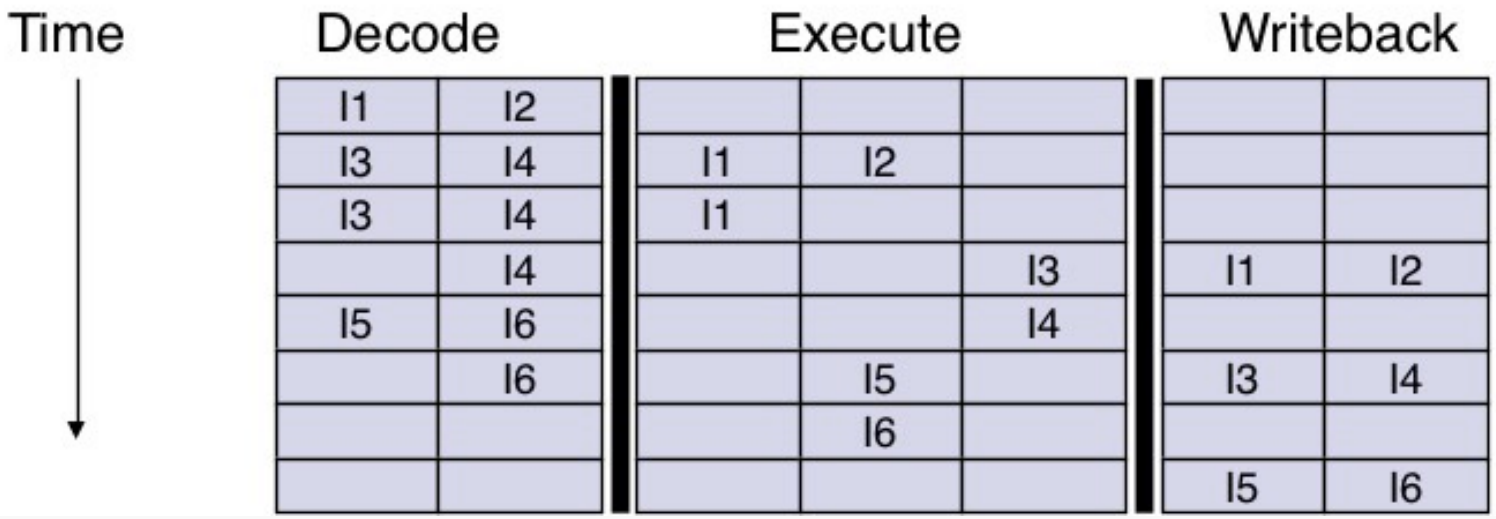


- 14 stage pipeline
- Branch mispredictions cost 17 cycles

Out-of-Order (O3) execution

- Example from: https://staff.fnwi.uva.nl/a.d.pimentel/aca/
- Assume a 3 stage execution in a pipeline that can issue two instructions, execute three instructions and write back two results every cycle… assume:
  - I1 requires 2 cycles to execute
  - I3 and I4 are in conflict for a functional unit
  - I5 depends on the value produced by I4
  - I5 and I6 are in conflict for a functional unit

In-order issue, in-order completion

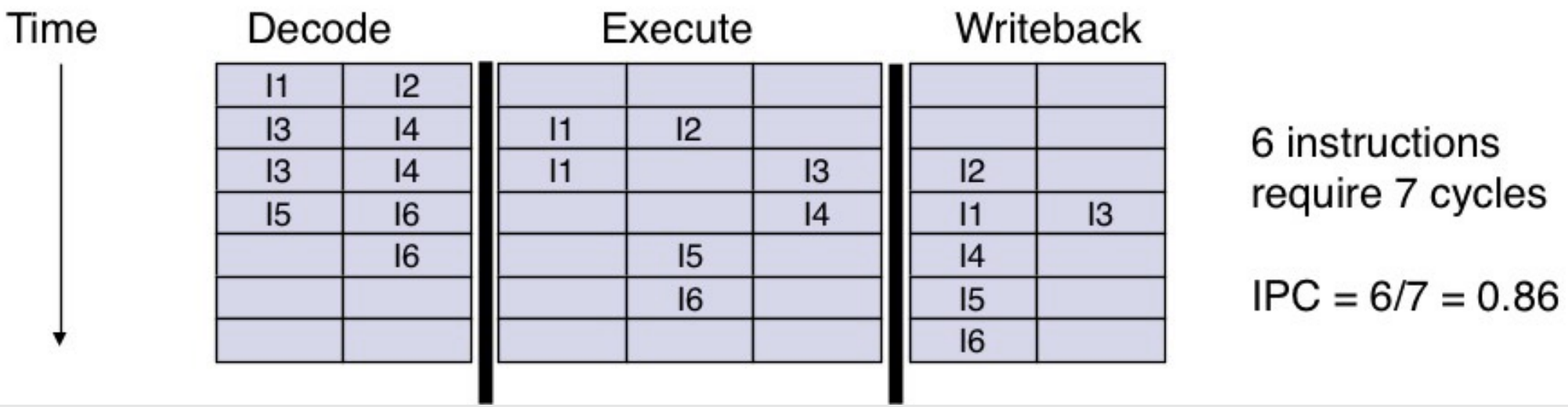


6 instructions require 8 cycles

IPC = 6/8 = 0.75

Out-of-order completion

- any number of instructions allowed in the execution stage up to the total number of pipeline slots (stages x functional units)
- instruction issue is not stalled when an instruction takes more than one cycle to complete



Out-of-order issue, out-of-order completion

- In-order issue stalls when the decoded instruction has:
  - a structural dependency, a true data dependency or an output dependency on an uncompleted instruction
  - (if two instructions write to the same register, they must complete in the correct order)
  - this is true even if instructions after the stalled one can execute
  - to avoid stalling, decode must be decoupled from execution

Out-of-order issue out-of-order completion

- Conceptually out-of-order issue decouples the decode/issue stage from instruction execution
  - it requires an **instruction window** between the decode and execute stages to buffer decoded or part pre-decoded instructions
  - this buffer serves as a pool of instructions giving the processor a look-ahead facility
  - instructions are issued from the buffer in any order, provided there are no resource conflicts or dependencies with executing instructions

Out-of-order issue out-of-order completion